SECTION 06 · PROVIDERS · GATEWAYS · LOCAL RUNTIMES

Jev-Like APIs Across Providers

The System One contract is spreading. Reach other providers' decision models by changing model or base_url, compare them on your own data, and choose a backend.

/v1/systemone | OpenRouter | Local servers | Agreement | Choosing

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

The Shared Request and Response Format

Request · POST /v1/systemone

```
{
  "model": "jev-latest",
  "state": "my card still hasn't come",
  "questions": {
    "intent": {
      "type": "choice",
      "instructions": "What does
          the customer need?",
      "criteria": {
        "card_arrival": "...",
        "change_pin": "...",
        "...8 more": "..."
      }}}
}
```

decision model

reads state + question once

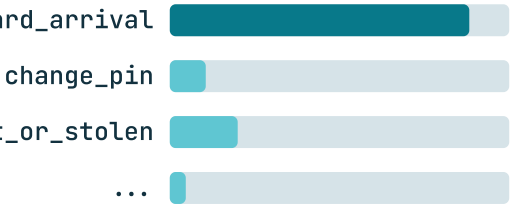
- card_arrival → score
- change_pin → score
- lost_or_stolen → score
- ... 7 more → score

softmax over YOUR options

no text is generated

Response · typed answers

```
"model": "<provider>/<name>-<version>"
"answers": {"intent": {
  "type": "choice",
  "choice": "card_arrival",
  "confidence": ...,
  "probabilities": {
```



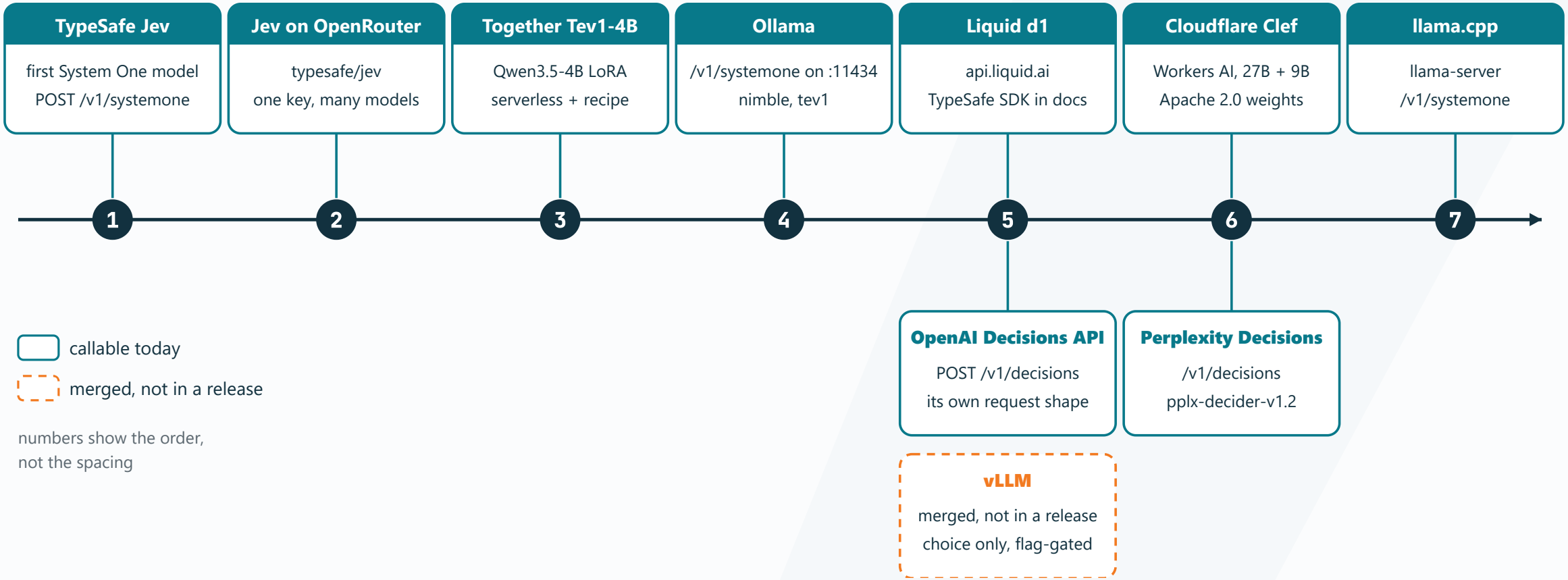
sum to 1

```
"usage": {"input_tokens": ...,
  "output_tokens": ...}
```

`noul`	-> P(yes), one number
`choice`	-> one key + a probability per option
`score`	-> score = sum of i x p(i)

Every backend in this section accepts this body. What differs: the host, the model name and the limits.

Timeline of Decision Model APIs



Each entry is confirmed on the provider's own page, post or release notes.

The Decision Model Landscape

Hosted APIs

TypeSafe
`Jev · api.typesafe.ai`

Liquid
`d1 · api.liquid.ai`

Perplexity
`pplx-decider · /v1/decisions`

Cloudflare Workers AI
`Clef, Clef-flash · /ai/run`

Together
Tev1-4B-experimental

Upstage
Solar Decide

OpenAI
`Decisions API · /v1/decisions`

Gateways

OpenRouter
`/api/v1/systemone`
`/api/alpha/decisions`

Vercel AI Gateway
AI SDK experimental_decide

LiteLLM proxy
`/typesafe/v1/systemone`

Local runtimes

Ollaya
`:11435 · Laya`

Ollama
`:11434 · nimble, tev1, clef`

llama.cpp
`llama-server · /v1/systemone`

SGLang
`/v1/systemone · /v1/decisions`

vLLM
merged, not in a release

Open weights

nimble 9B
Bespoke Labs

Tev1 4B, 0.8B
Together

Laya 322M to 421M
Convai Innovations

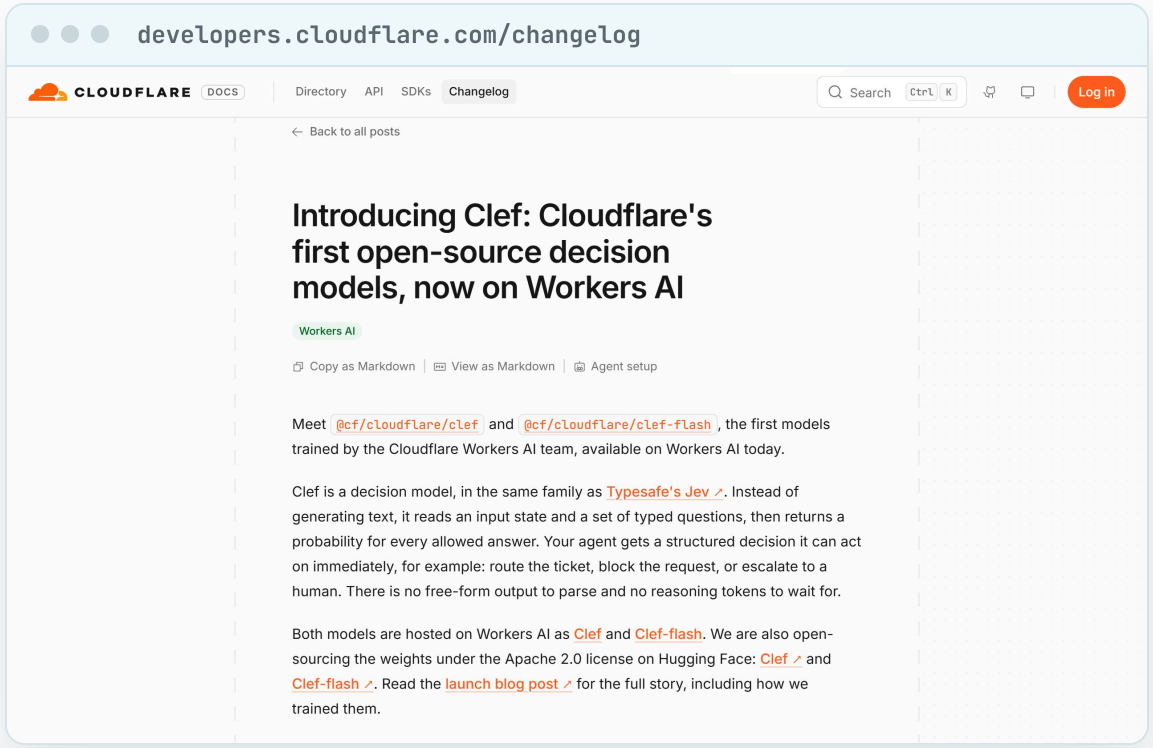
Kev 0.8B to 27B
Jared Palmer

Clef 27B, Clef-flash 9B
Cloudflare

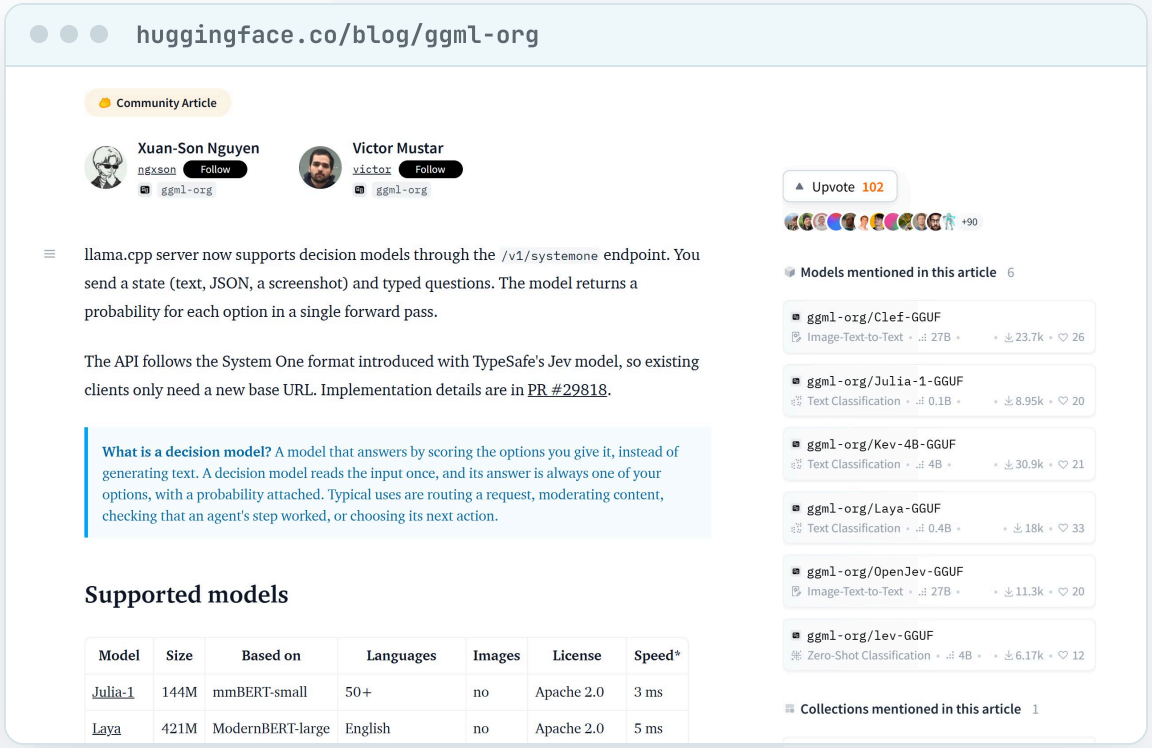
pplx-decider 27B
Perplexity

decider
Mapika

Providers That Claim Jev Compatibility



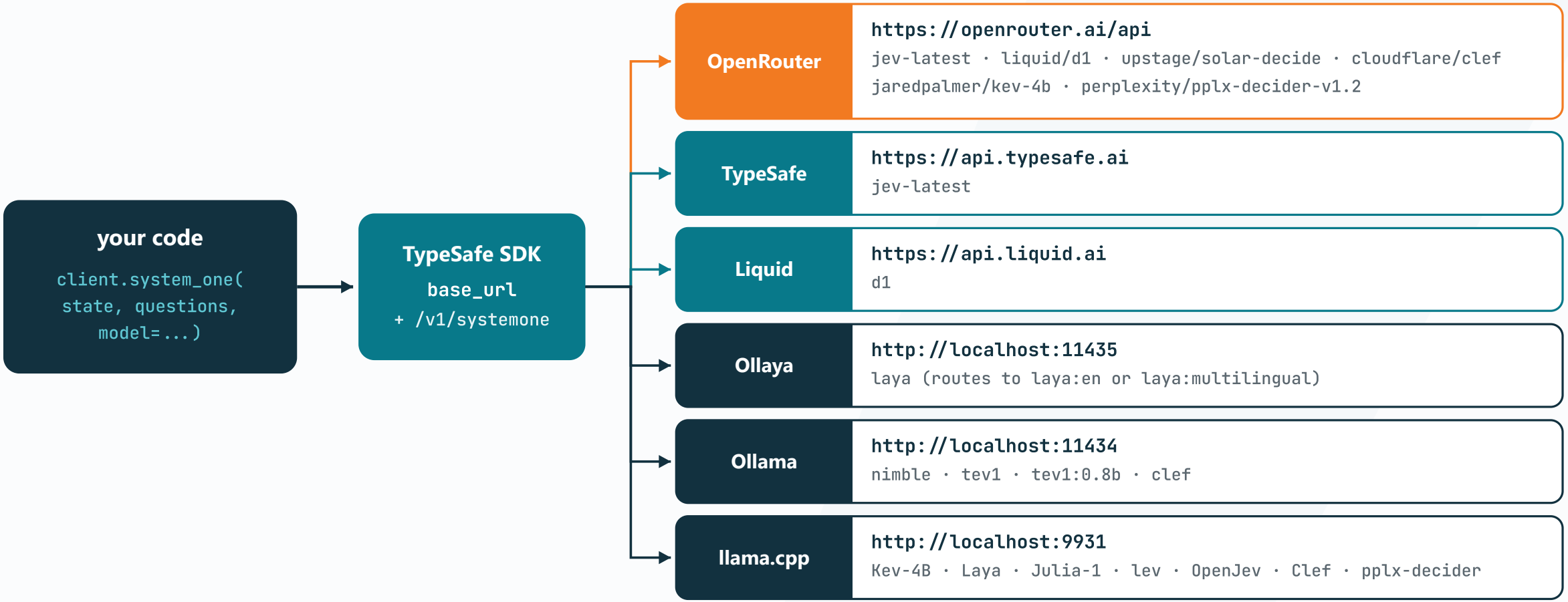
Cloudflare changelog: Clef is in the same family as Jev, typed questions in, probabilities out.



llama.cpp: /v1/systemone in llama-server, existing clients only need a new base URL.

Compatibility is a claim until your own eval set passes on the new backend.

One Contract, Many Backends



MODEL= PICKS THE MODEL, BASE_URL= PICKS THE SERVER

Switching Providers With One Parameter

```python
client = TypeSafeClient()     # OpenRouter from .env

client.system_one(msg, q, model="jev-latest")
client.system_one(msg, q, model="liquid/d1")
client.system_one(msg, q, model="cloudflare/clef")

local = TypeSafeClient(
    base_url="http://localhost:11434",
    api_key="local")
local.system_one(msg, q, model="nimble")
```

ALIAS IN, DATED VERSION OUT

jev-latest	typesafe/jev-1.14-20261014
liquid/d1	liquid/d1-20260930
jaredpalmer/kev-4b	jaredpalmer/kev-4b-20260924
perplexity/pplx-decider-v1.2	perplexity/pplx-decider-v1.2-20261016

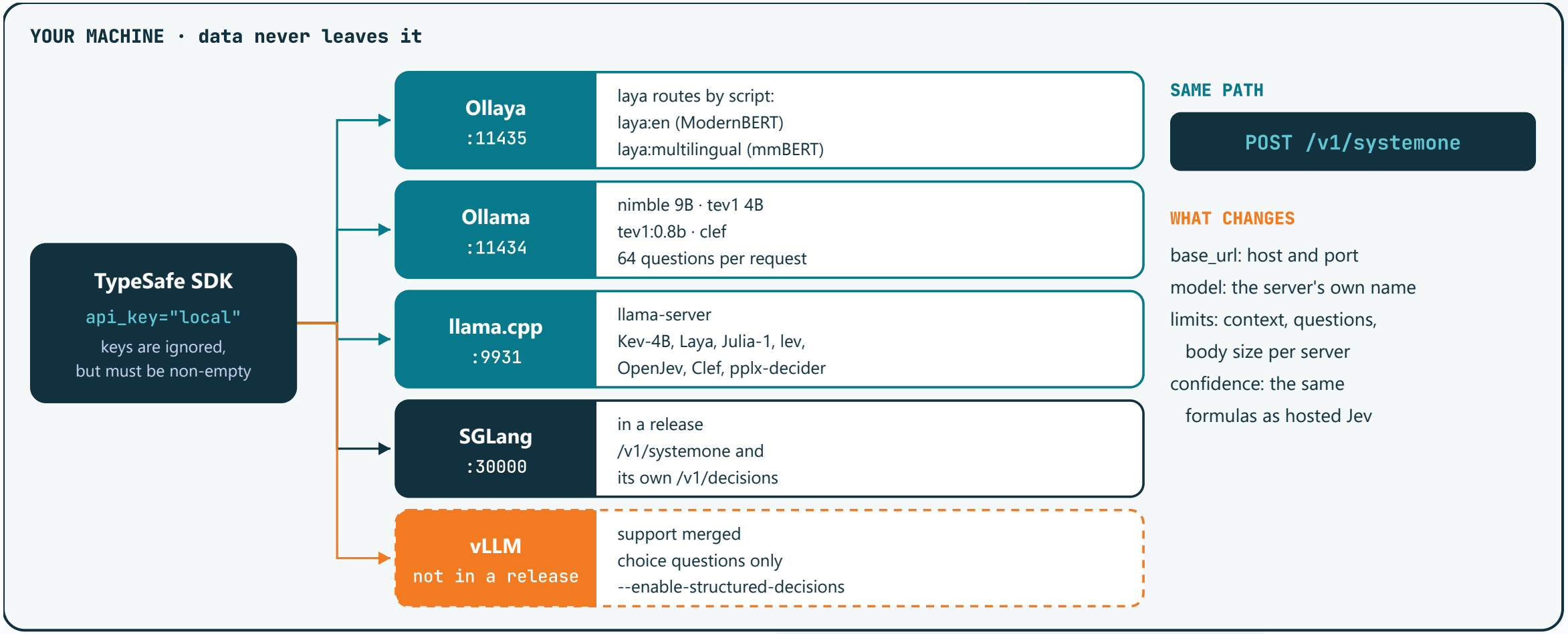
model= what you ask for

r.model what actually answered: log it

an alias can move to a new version;
a pinned ID plus your eval set catches the change

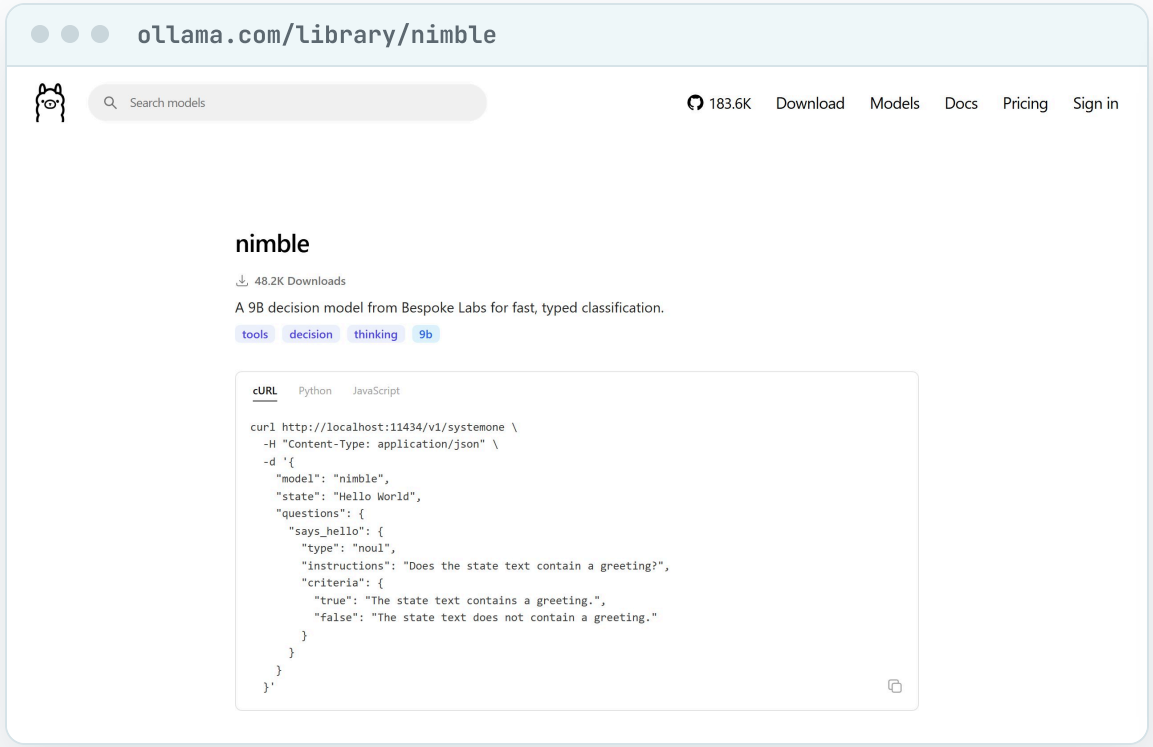
at Perplexity direct, r.model echoes the name you sent

The Same Contract, Locally

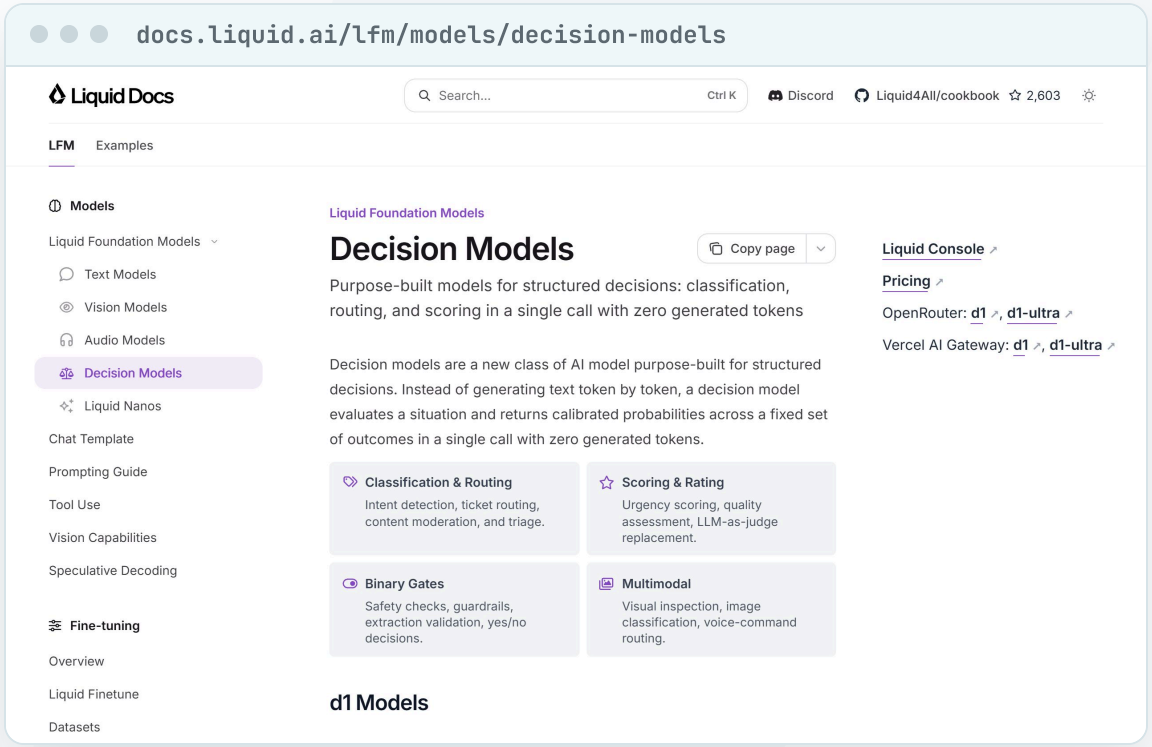


THE REQUEST BODY IS THE SAME IN BOTH DOCS

One Body, Local or Hosted



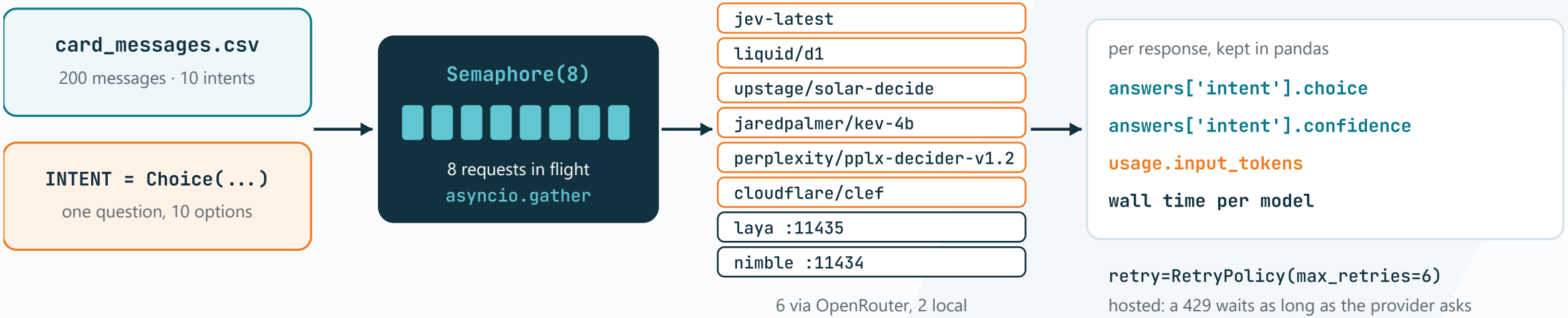
Ollama: curl to localhost:11434/v1/systemone with model, state, questions.



Liquid: the docs call d1 with the TypeSafe SDK and Liquid's own base URL.

Same three fields: **model**, **state**, **questions**.

A Fair Benchmark Harness



THE TABLE: ONE ROW PER MODEL

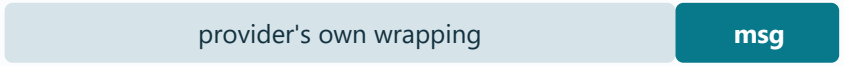
model	accuracy	mean_confidence	tokens_per_message	seconds_for_200	usd_per_million_messages
jev-latest					
liquid/d1					
...					
nimble					NaN (your GPU)

Bars are placeholders: the notebook fills the numbers, and they change from run to run.

Cost per Message, Not per Token

THE SAME MESSAGE AT TWO PROVIDERS (example numbers)

Model A



300 input tokens

300 tokens per message	x	$0.04 per million tokens	=	$12.00 per million messages

Model B



60 input tokens

60 tokens per message	x	$0.09 per million tokens	=	$5.40 per million messages

WHY THE COUNTS DIFFER

own prompt template
state and questions are wrapped in each provider's own text

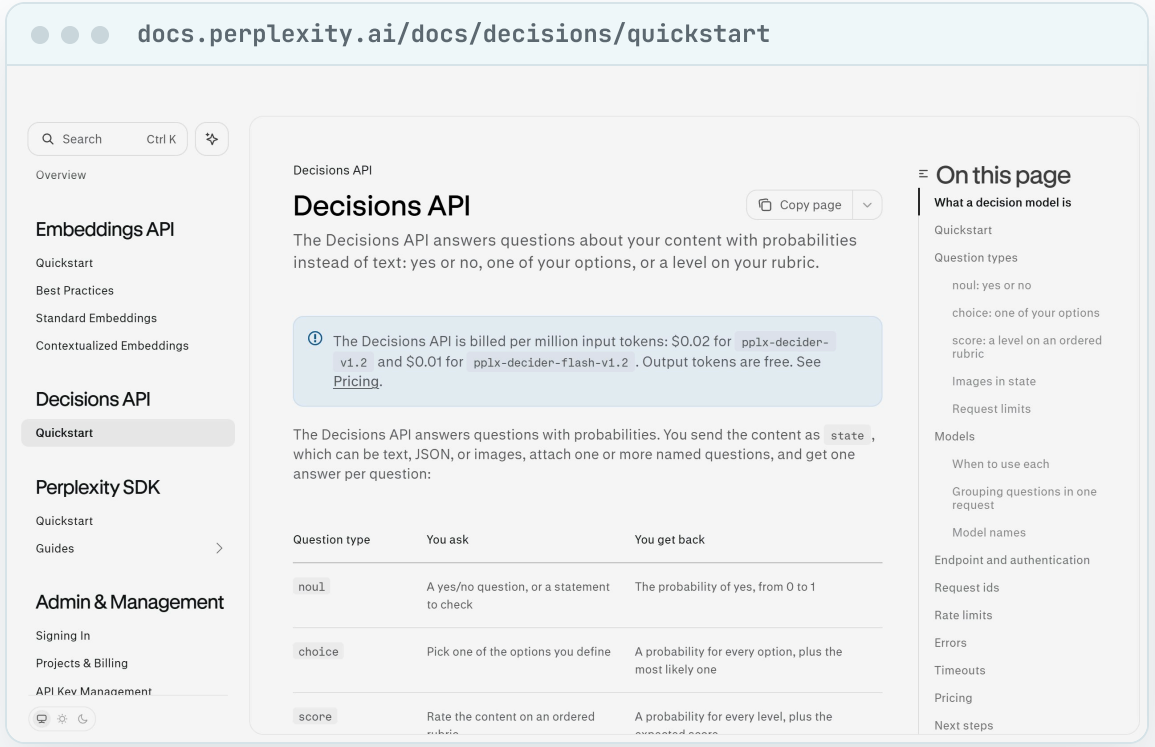
own tokenizer
the same words split into a different number of tokens

questions count too
instructions, labels and descriptions are input tokens

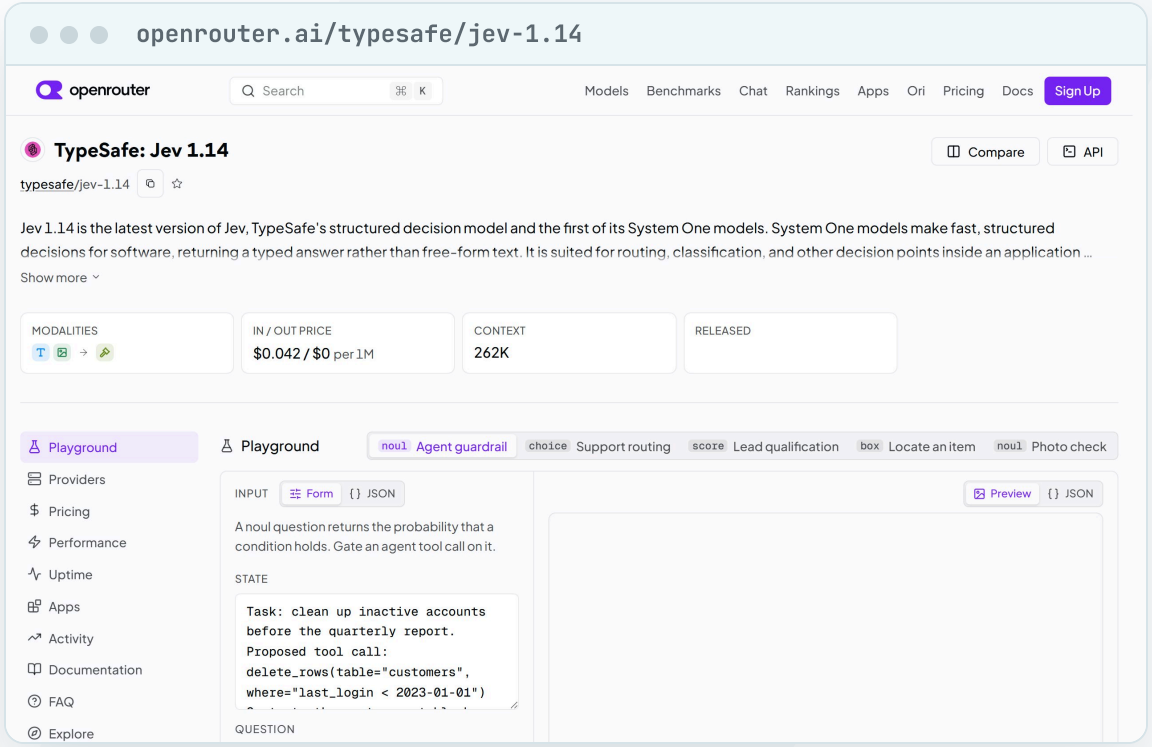
tokens per message x price per million tokens = dollars per million messages

the cheaper price per token costs more per message here:
count tokens on your own messages at each provider, then compare

Where to Find Each Provider's Price



Perplexity: $0.02 per million input tokens for pplx-decider-v1.2, output free; state can be text, JSON or images.



OpenRouter: Jev 1.14 at $0.042 in, $0 out per million tokens; 32K per question by default.

Billing on input tokens with free output is the common pattern across these APIs.

Published Prices and Limits

model ID on OpenRouter	input $ per million tokens (output: $0 for all)	context, tokens
typesafe/jev-1.14	$0.042	32K per question
liquid/d1	$0.02	65,536
upstage/solar-decide	$0.05	524,288
jaredpalmer/kev-4b	$0.042	8,192
perplexity/pplx-decider-v1.2	$0.02	262,144
cloudflare/clef	$0.042	16,384
cloudflare/clef-flash	$0.021	16,384
togethercomputer/tev1-4b-experimental	$0.042	32,768

Source: OpenRouter GET /api/v1/models/<id>/endpoints, the cheapest endpoint: pricing.prompt and context_length.
Local models have no token price: nimble on Ollama fits an 8,192-token prompt and 64 questions per request.

ONE ROW PER MESSAGE, ONE COLUMN PER MODEL

Where Models Disagree

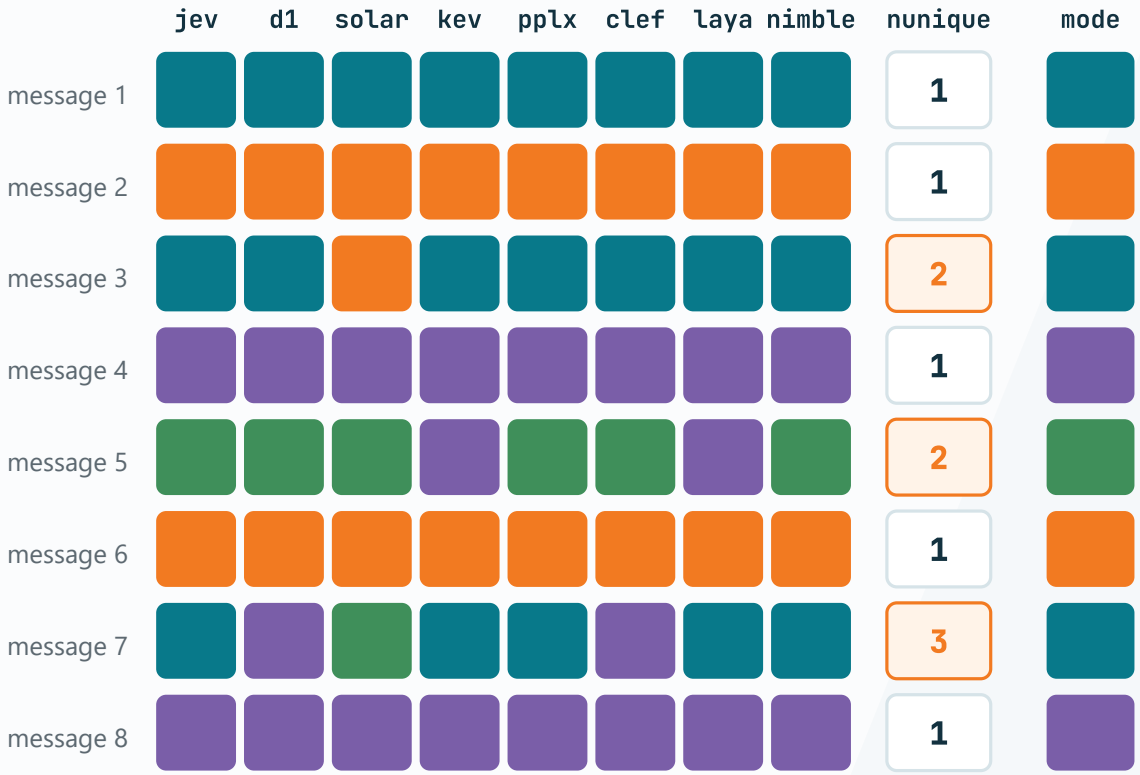


illustration: each colour is one intent

nunique == 1

every model gave the same answer

candidates for automation

nunique > 1

the models split

ambiguous text, a typo, or two intents at once: send to a person

```
pd.crosstab(all_agree,
  majority == intent)
```

Same Base Model, Same Mistakes

Qwen3.5-4B	**Tev1-4B** Together · LoRA
	Kev-4B LoRA + pointer head
Qwen3.5-9B	**nimble** Bespoke Labs · LoRA
Qwen3.8-27B	**pplx-decider-v1.2** Perplexity
	Clef Cloudflare
Qwen3.5-2B	**Strands Decider 2B** Strands Agents · LoRA
Solar Mini 4	**Solar Decide** Upstage
ModernBERT / mmBERT	**Laya** Convai · encoder

Independent models agree

two separate readings of the message point the same way: strong evidence



four different bases

Siblings agree

fine-tunes of one base may share its blind spots: agreement counts less



one base, four heads

base not named in their pages: Jev, Liquid d1

SOME PROVIDERS KEEP THE IDEA BUT NOT THE PATH

Same API Path or Own API

Same path: change base_url	
TypeSafe	/v1/systemone
OpenRouter	/api/v1/systemone
Liquid	/v1/systemone
Ollama, Ollaya, llama.cpp	/v1/systemone
SGLang	/v1/systemone
LiteLLM proxy	/typesafe/v1/systemone

Own path: same idea, other call	
Perplexity	POST /v1/decisions
Cloudflare Workers AI	env.AI.run(), /ai/run
OpenRouter native	/api/alpha/decisions
SGLang native	/v1/decisions
Vercel AI SDK	experimental_decide
OpenAI	POST /v1/decisions

THREE WAYS TO REACH AN OWN-PATH PROVIDER

raw HTTP

```
httpx.post(url, json={model, state, questions})
```

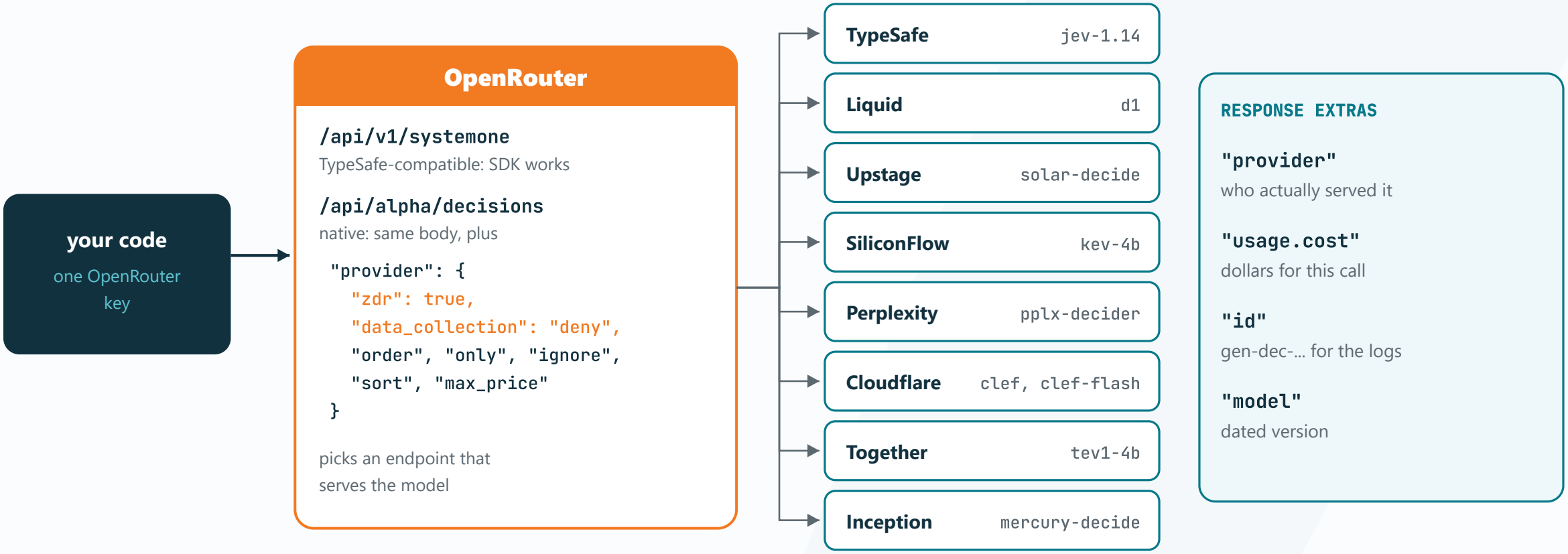
their SDK or binding

Cloudflare env.AI.run, Vercel AI SDK

a gateway

OpenRouter serves pplx-decider and Clef on /v1/systemone

How a Gateway Routes a Request

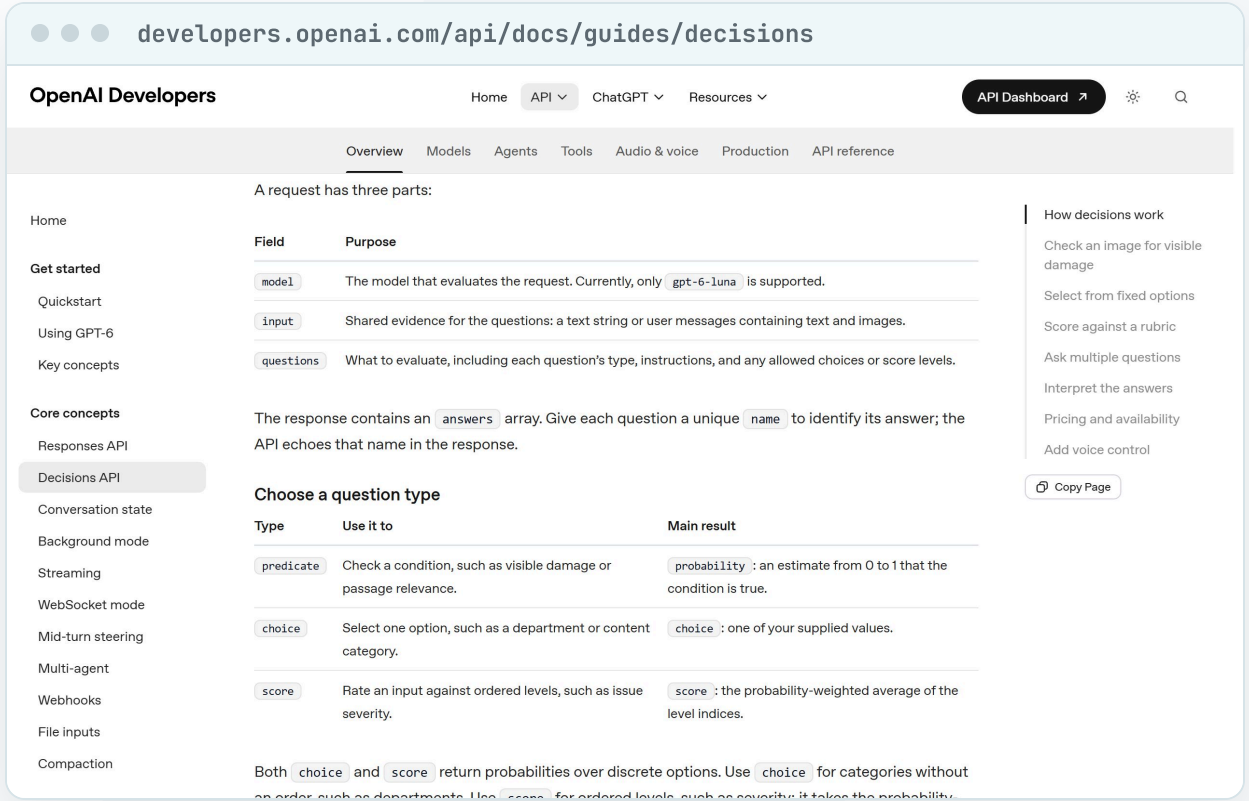


zdr: only endpoints with zero data retention. If no endpoint matches the rules, the call fails.

OpenAI's Own Decisions API

System One	OpenAI Decisions
path POST /v1/systemone	POST /v1/decisions
model jev-latest, d1, ...	gpt-6-luna
evidence "state"	"input"
questions object keyed by name	array, each with a name
yes or no "type": "noul"	"type": "predicate"
answers object keyed by name	array of answers
images in the state	image parts in input

Own schema: not System One compatible, so it needs its own client call.



OpenAI Decisions guide: model, input and questions, answers come back as an array.

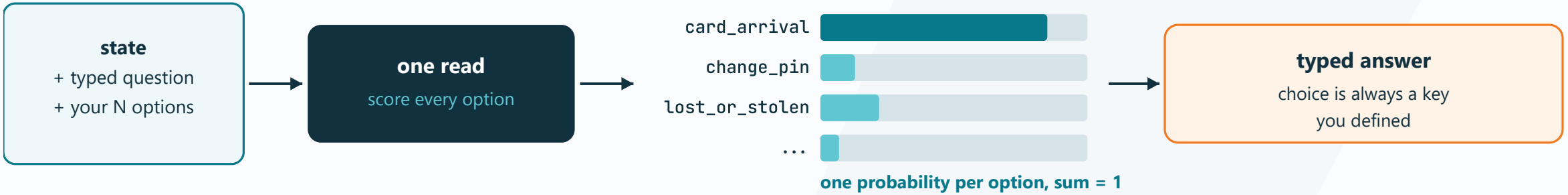
Chat Model vs Decision Model

GENERATION · chat model



logprobs: a probability for each generated token over the whole vocabulary, not one probability per option you defined; options split into several tokens

DECISION · System One model



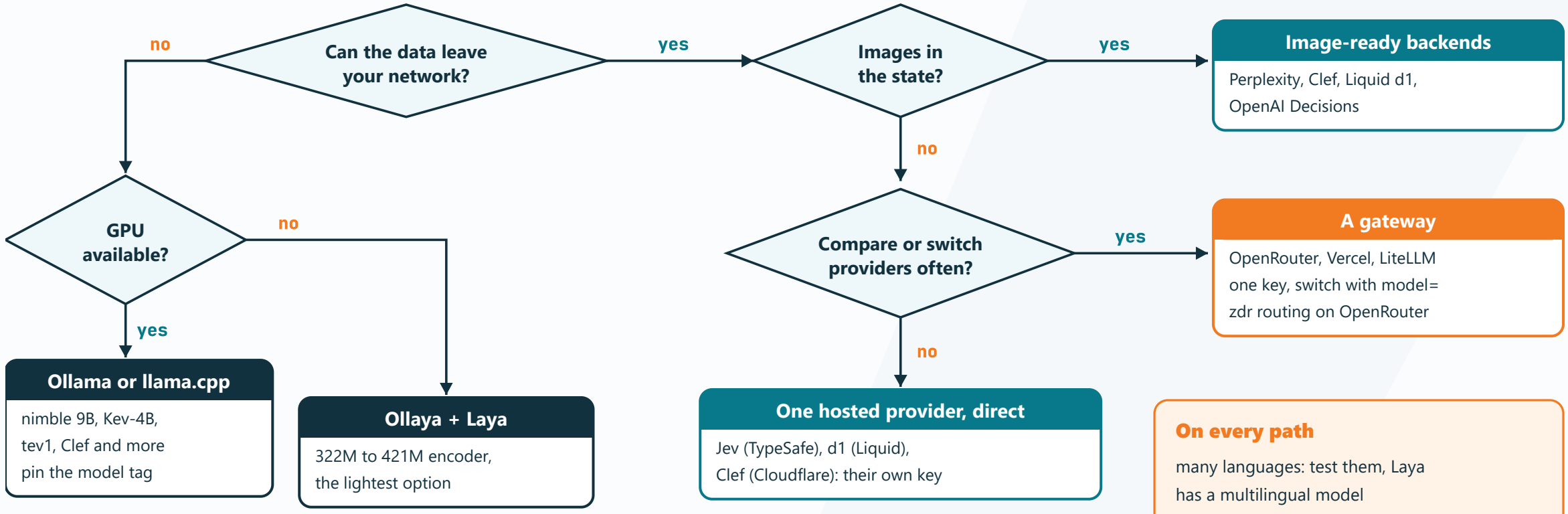
output tokens are free: billing is on input tokens

Features That Look Like a Decision API

	answer is one of your options	probability for every option	sums to 1 over your options	trained for calibration	no text to parse
Prompt + parse the reply	✗	✗	✗	✗	✗
Logprobs	✗	◑	✗	✗	✗
JSON mode	✗	✗	✗	✗	◑
Structured outputs (enum)	✓	✗	✗	✗	◑
Decision endpoint	✓	✓	✓	✓ *	✓

* trained for it, but calibration differs per model: check confidence against accuracy on your own labelled data.
half circle: logprobs cover only the tokens of the reply; JSON still has to be parsed.

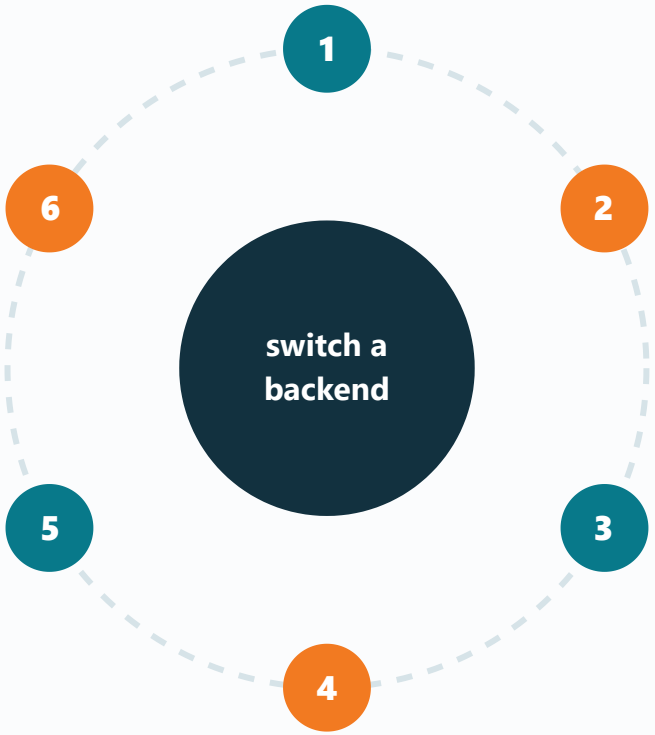
How to Choose a Backend



On every path

many languages: test them, Laya has a multilingual model
volume: tokens per message x price
accuracy: your own labelled set

A Portability Checklist



1. **Pin the model ID**
   log r.model on every call
2. **Keep an eval set**
   labelled CSV, your messages
3. **Re-run on every switch**
   accuracy, confidence, tokens
4. **Compare per message**
   tokens x price, wall time
5. **Check the limits**
   context, questions, body size
6. **Re-set thresholds**
   confidence differs per model

Base URLs and Model IDs

backend	base_url or endpoint	model IDs
OpenRouter	`https://openrouter.ai/api`	`jev-latest`, `liquid/d1`, `upstage/solar-decide`, `jaredpalmer/kev-4b`, `perplexity/pplx-decider-v1.2`, `cloudflare/clef`
OpenRouter native	`POST /api/alpha/decisions`	same model IDs, + provider routing
TypeSafe	`https://api.typesafe.ai`	`jev-latest`
Liquid	`https://api.liquid.ai`	`d1`
Perplexity	`POST https://api.perplexity.ai/v1/decisions`	`pplx-decider-v1.2` (own path, raw HTTP)
Cloudflare	`env.AI.run()` or `REST /ai/run`	`@cf/cloudflare/clef`, `@cf/cloudflare/clef-flash`
Ollaya	`http://localhost:11435`	`laya`, `laya:en`, `laya:multilingual`
Ollama	`http://localhost:11434`	`nimble`, `tev1`, `tev1:0.8b`
llama.cpp	`http://localhost:9931`	the GGUF loaded by `llama-server`
OpenAI	`POST https://api.openai.com/v1/decisions`	`gpt-6-luna` (own request shape)

SECTION 06 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie